

Organización de Computadoras Guía de Ejercicios

No 5 - ASM RISC-V

1. Entorno de desarrollo: configuración del toolchain y ensamblador

- a. **Configuración del entorno de desarrollo-** Instalar Extensiones Necesarias en VS Code
 - i. Abre Visual Studio Code.
 - ii. Ve a la pestaña de Extensiones (Ctrl+Shift+X o el ícono de extensiones en la barra lateral).
 - iii. Busca e instala las siguientes extensiones:
 - 1. C/C++: Para soporte de sintaxis y resaltado de código.
 - 2. RISC-V Support: Para soporte de ensamblador RISC-V.
 - 3. Makefile Tools: Para trabajar con archivos Makefile.
 - 4. ESPTool: Para cargar el firmware en el ESP32-C3 (opcional, pero útil).

b. Crear tu proyecto

- i. Ahora que el entorno está configurado, puedes crear tu proyecto en cualquier carpeta. Por ejemplo:
- ii. Crea una carpeta para tu proyecto, puede ser directamente desde la terminal (consola)

```
mkdir -p ~/mis_proyectos/mi_proyecto_esp32c3_asm  
cd ~/mis_proyectos/mi_proyecto_esp32c3_asm
```

c. Crear la Estructura del Proyecto

- i. Abre una carpeta nueva en VS Code (Archivo > Abrir Carpeta).
- ii. Dentro de la carpeta, crea los siguientes archivos y directorios:

```
mi_proyecto_asm/  
├── Makefile  
├── linker.ld  
├── main.S  
└── .vscode/  
    └── tasks.json
```

d. Configurar Tareas en VS Code

- i. Crea el directorio .vscode:

```
mkdir .vscode
```
- ii. Navega al directorio .vscode:

```
cd .vscode
```
- iii. Crea el archivo tasks.json:

```
touch tasks.json
```
- iv. Escribe lo siguiente en tasks.json:

```
{  
  "version": "2.0.0",  
  "tasks": [  
    {  
      "label": "Build Project",  
      "type": "shell",  
      "command": "make",  
      "args": ["all", "-j4"],  
      "group": {  
        "kind": "build",
```

```

        "isDefault": true
    },
    "problemMatcher": [
        {
            "owner": "assembler",
            "fileLocation": ["relative", "${workspaceFolder}"],
            "pattern": {
                "regexp":
"^(.*):(\\d+):(\\d+):\\s+(warning|error):\\s+(.*)$",
                "file": 1,
                "line": 2,
                "column": 3,
                "severity": 4,
                "message": 5
            }
        }
    ],
    "detail": "Compila el proyecto en assembly para ESP32-C3
usando 4 jobs en paralelo",
    "presentation": {
        "reveal": "always",
        "panel": "dedicated"
    }
},
{
    "label": "Clean Project",
    "type": "shell",
    "command": "make",
    "args": ["clean"],
    "group": "build",
    "problemMatcher": [],
    "detail": "Limpia los archivos generados",
    "presentation": {
        "reveal": "always",
        "panel": "dedicated"
    }
},
{
    "label": "Flash ESP32-C3",
    "type": "shell",
    "command": "make",
    "args": ["flash"],
    "group": "build",
    "problemMatcher": [],
    "detail": "Flashea el firmware en el ESP32-C3",
    "presentation": {
        "reveal": "always",
        "panel": "dedicated",
        "clear": true
    }
},
{
    "label": "Monitor Serial",
    "type": "shell",
    "command": "idf.py",

```

```

        "args": ["monitor"],
        "group": "test",
        "problemMatcher": [],
        "detail": "Inicia el monitor serial para ver la salida del
ESP32-C3",
        "presentation": {
            "reveal": "always",
            "panel": "new",
            "focus": true
        }
    },
    {
        "label": "Build + Flash",
        "dependsOrder": "sequence",
        "dependsOn": ["Build Project", "Flash ESP32-C3"],
        "group": "build",
        "problemMatcher": [],
        "detail": "Compila y flashea el firmware en un solo paso"
    }
]
}

```

- v. Esto te permitirá compilar y limpiar tu proyecto directamente desde VS Code.
- vi. Guarda y cierra el archivo.
- vii. Regresa a la carpeta principal del proyecto:

```
cd ..
```

e. Escribir el Código en Ensamblador

El archivo main.S contendrá el código en ensamblador que suma dos números.

```

.global _start

.section .vectors, "ax"
_start:
    j app_main      # Salto a la función principal

.section .text
app_main:
    # Inicialización del stack pointer
    la sp, _stack_end

    # Operaciones aritméticas básicas
    li a0, 15        # a0 = 15
    li a1, 7          # a1 = 7

    # Suma: a2 = a0 + a1
    add a2, a0, a1    # a2 = 22

    # Resta: a3 = a0 - a1
    sub a3, a0, a1     # a3 = 8

    # Operaciones lógicas
    and a4, a0, a1     # AND: 15 & 7 = 7
    or a5, a0, a1      # OR: 15 | 7 = 15
    xor a6, a0, a1     # XOR: 15 ^ 7 = 8

```

```

        # Bucle infinito
        j .

.section .text
.global _start

_start:
    #Aca va nuestro código, empecemos con el punto 1 de la guia
    "Guía Práctica Programacion nº 1 – ASM"

```

f. **Crear el Script de Enlazado (linker.ld)**

El archivo linker.ld define cómo se organizan las secciones de memoria en el ESP32-C3.

```

MEMORY {
    iram (rwx) : org = 0x40380000, len = 0x20000 /* IRAM del
ESP32-C3 */
    dram (rw) : org = 0x3FC80000, len = 0x20000 /* DRAM del
ESP32-C3 */
}

ENTRY(_start)

SECTIONS {
    /* Sección de vectores (opcional para ASM puro) */
    .vectors : {
        *(.vectors)
    } > iram

    /* Sección de código principal */
    .text : ALIGN(4) {
        *(.text)
        *(.text.*)
        *(.gnu.linkonce.t.*)
        *(.stub)
    } > iram

    /* Sección de datos de solo lectura */
    .rodata : ALIGN(4) {
        *(.rodata)
        *(.rodata.*)
        *(.gnu.linkonce.r.*)
    } > dram

    /* Sección de datos inicializados */
    .data : ALIGN(4) {
        _data_start = .;
        *(.data)
        *(.data.*)
        *(.gnu.linkonce.d.*)
        _data_end = .;
    } > dram

    /* Sección BSS (datos no inicializados) */
    .bss : ALIGN(4) {

```

```

    _bss_start = .;
    *(.bss)
    *(.bss.*)
    *(.gnu.linkonce.b.*)
    *(COMMON)
    _bss_end = .;
} > dram

/* Configuración del stack */
.stack (NOLOAD) : ALIGN(16) {
    _stack_start = .;
    . += 0x2000; /* 8KB de stack */
    _stack_end = .;
} > dram

/* ===== */
/* SECCIONES DE DEPURACIÓN */
/* ===== */

/* Secciones DWARF 1 */
.debug      0 : { *(.debug) }
.line       0 : { *(.line) }

/* Secciones DWARF 2 */
.debug_info  0 : { *(.debug_info) }
.debug_abbrev 0 : { *(.debug_abbrev) }
.debug_line  0 : { *(.debug_line) }
.debug_frame 0 : { *(.debug_frame) }
.debug_str    0 : { *(.debug_str) }
.debug_loc    0 : { *(.debug_loc) }
.debug_macinfo 0 : { *(.debug_macinfo) }

/* Secciones DWARF 3 */
.debug_pubnames 0 : { *(.debug_pubnames) }
.debug_pubtypes 0 : { *(.debug_pubtypes) }

/* Secciones DWARF 4 */
.debug_aranges 0 : { *(.debug_aranges) }
.debug_ranges  0 : { *(.debug_ranges) }

/* Secciones DWARF 5 */
.debug_addr     0 : { *(.debug_addr) }
.debug_line_str 0 : { *(.debug_line_str) }
.debug_loclists 0 : { *(.debug_loclists) }
.debug_rnglists 0 : { *(.debug_rnglists) }

/* Símbolos para el debugger */
.comment      0 : { *(.comment) }
.note.gnu.build-id : { *(.note.gnu.build-id) }

/* Variables para el debugger */
_iram_start = ORIGIN(iram);
_iram_end = ORIGIN(iram) + LENGTH(iram);
_dram_start = ORIGIN(dram);
_dram_end = ORIGIN(dram) + LENGTH(dram);

```

```
}
```

g. Crear el Makefile

El Makefile automatiza el proceso de compilación y enlazado

```
# Toolchain configuration
CC = riscv32-esp-elf-gcc
AS = riscv32-esp-elf-as
LD = riscv32-esp-elf-ld
OBJCOPY = riscv32-esp-elf-objcopy
SIZE = riscv32-esp-elf-size
ESPTOOL = esptool.py

# Target configuration
TARGET = main
MCU = esp32c3

# Flags
CFLAGS = -march=rv32imc -mabi=ilp32 -nostdlib
ASFLAGS = -march=rv32imc -mabi=ilp32
LDFLAGS = -T linker.ld -nostartfiles -nostdlib --gc-sections
DEBUG_FLAGS = -g -gdwarf-2
ASFLAGS += $(DEBUG_FLAGS)

# Source files
SRCS = $(TARGET).S

# Build directory
BUILD_DIR = build

.PHONY: all clean flash

all: $(BUILD_DIR)/$(TARGET).bin

$(BUILD_DIR)/$(TARGET).o: $(SRCS) | $(BUILD_DIR)
    $(AS) $(ASFLAGS) -o $@ $<

$(BUILD_DIR)/$(TARGET).elf: $(BUILD_DIR)/$(TARGET).o
    $(LD) $(LDFLAGS) -o $@ $<
    $(SIZE) $@

$(BUILD_DIR)/$(TARGET).bin: $(BUILD_DIR)/$(TARGET).elf
    $(OBJCOPY) -O binary $< $@

$(BUILD_DIR):
    mkdir -p $@

flash: $(BUILD_DIR)/$(TARGET).bin
    $(ESPTOOL) -p /dev/ttyUSB0 -b 460800 --before default_reset --after
    hard_reset \
    write_flash --flash_mode dio --flash_freq 80m --flash_size 2MB \
    0x10000 $(BUILD_DIR)/$(TARGET).bin

clean:
    rm -rf $(BUILD_DIR)
```

h. Compilar y Generar el Binario

En la terminal, navega hasta el directorio de tu proyecto y ejecuta:

```
make
```

Esto generará un archivo main.bin que puedes cargar en tu ESP32-C3.

i. Cargar el firmware en el ESP32-C3

Puedes usar esptool.py para cargar el binario en tu ESP32-C3. Asegúrate de tener esptool instalado y luego ejecuta:

```
esptool.py --chip esp32c3 --port /dev/ttyUSB0 --baud 115200  
write_flash 0x0 main.bin
```

Ajusta el puerto (/dev/ttyUSB0) según tu configuración.

j. Ejecutar el Programa

Una vez cargado el binario, el ESP32-C3 ejecutará el código que hayas escrito.

k. Probar funcionamiento del Programa

- i. Verificación básica con esptool (Solo confirma que se programó)

```
esptool.py --port /dev/tty.usbserial-1410 read_flash 0x0 0x100 main_read.bin  
md5sum main.bin main_read.bin
```

- ii. Ver el contenido en hexadecimal

```
hexdump -C main_read.bin | head -20
```

- iii. Visualizar secciones

```
riscv32-esp-elf-readelf -S main.elf
```

- iv. Depuración

1. Inicia OpenOCD en una terminal:

```
openocd -f board/esp32c3-builtin.cfg
```

2. En otra terminal, inicia GDB:

```
riscv32-esp-elf-gdb main.elf
```

3. Dentro de GDB:

```
target remote :3333
```

```
break _start # Punto de entrada de tu programa
```

```
continue # Continúa hasta el breakpoint
```

```
info registers a2 # Muestra el valor del registro
```

4. Salida esperada

```
a2      0xf      15 # 0xf es 15 en hexadecimal
```

2. ¿Que información te da cada una de las opciones de 1.k.?

3. Primeros programas en ensamblador para ESP32-C3 (manejo de registros)

- a. Implementa el ejercicio 7 de la guía “Guía Práctica Programacion nº 1 – ASM” en el ESP32-C3.

- b. Implementa el ejercicio 9 de la guía “Guía Práctica Programacion nº 1 – ASM” en el ESP32-C3.

- c. Implementa el ejercicio 1 (solo la suma) de la guía “Guía Práctica Programacion nº 2 – ASM” en el ESP32-C3.

4. Crea el código para resolver los siguientes problemas, hazlo en el simulador web

- a. Ejercicio 1:

- i. Simula una operación de copia de "inputs" de un sensor a un buffer de entrada para inferencia.
- ii. Se te da un arreglo de 8 números enteros simulando datos de entrada de sensores. Se deben transferir estos datos a un segundo arreglo (buffer de procesamiento). Hazlo utilizando un bucle y registros temporales.

iii. Datos:

```
.data
input:  .word 10, 20, 30, 40, 50, 60, 70, 80
buffer: .space 32
```

b. Ejercicio 2:

- i. Imaginemos que tenemos una pequeña matriz de pesos para una red neuronal (por ejemplo, una capa fully connected) y queremos copiar esos valores a registros para su posterior uso.
- ii. Dado el siguiente arreglo en memoria:

```
.data
weights: .word 10, 20, 30, 40
```

c. Ejercicio 3:

- i. En una red, queremos identificar si alguna activación supera cierto umbral. Vamos a recorrer un arreglo de activaciones y verificar si alguna supera 50.

```
.data
activations: .word 45, 60, 22, 53
threshold: .word 50
result: .word 0
```

ii. Instrucciones:

- Recorrer el arreglo activations.
- Si alguna activación es mayor al umbral, escribir 1 en result y salir del programa.